

UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
DISCIPLINA: PROGRAMAÇÃO 2 (P2)

ALUNA: RAYSSA

RELATÓRIO DO PROJETO JACKUT

Milestones 1 e 2 — User Stories 1 a 9

MACEIÓ, ALAGOAS

1. INTRODUÇÃO

Este relatório apresenta a descrição do projeto Jackut, uma simulação de rede social inspirada no Orkut, desenvolvida ao longo de duas fases na disciplina de Programação 2. Ele cobre a avaliação do design adotado e o refatoramento realizado entre os milestones, com foco nas decisões arquiteturais que sustentam as user stories de 1 a 9.

O documento está organizado da seguinte maneira: na seção “Avaliação”, são destacados os pontos positivos e negativos do design, ressaltando as consequências dessas escolhas na adição de novas funcionalidades. Na seção “Refatoramento e Desenvolvimento Realizado”, descreve-se o refatoramento necessário para corrigir as penalidades do milestone 1 e implementar as funcionalidades do milestone 2, bem como os padrões utilizados. Por fim, a seção “Conclusão” apresenta comentários gerais e uma avaliação objetiva referente à qualidade dos deliverables.

2. AVALIAÇÃO

De maneira geral, o projeto evoluiu de um design inicial penalizado por concentração de responsabilidades para uma arquitetura em camadas, na qual a inserção de novas funcionalidades tornou-se uma tarefa relativamente direta. Ainda assim, uma análise mais refinada revela pontos que poderiam ser melhor explorados, sobretudo no uso de herança e polimorfismo de domínio. Estas observações são apresentadas com mais detalhes nas subseções a seguir.

2.1 VIRTUDES DO PROJETO

Em termos estruturais, o projeto é bastante organizado. A divisão entre os pacotes apresenta coerência: exceptions, models, repository, service e persistence, além do pacote raiz. Cada pacote tem sua documentação interna (package-info.java em Javadoc), bem como suas classes, métodos, parâmetros e exceções documentados.

A classe Facade, usada como fachada para a invocação dos testes de aceitação com a ferramenta EasyAccept, é enxuta e não apresenta regras de negócio dentro dela, como preconiza um bom design. Ela apenas adapta a interface esperada pelos scripts (assinaturas e conversão de valores booleanos para as strings “true”/“false”) e delega cada chamada ao Sistema.

Destaca-se como ponto positivo a separação entre a Facade e o Sistema. Enquanto a Facade isola o framework de testes, o Sistema atua como orquestrador: resolve a sessão e o usuário envolvidos em cada operação e encaminha a chamada ao serviço responsável, sem concentrar lógica de negócio. As regras efetivas residem em sete classes de serviço (AmizadeService, RecadoService, ComunidadeService, MensagemService, RelacionamentoService, RemocaoService) e nos gerenciadores de sessão e persistência.

O tratamento de erros foi modelado por meio de um vocabulário de exceções específico do domínio. Existe uma classe base, JackutException, que estende RuntimeException, e vinte e duas subclasses, cada uma em arquivo próprio, fixando no construtor a mensagem esperada

pelos testes. Isso centraliza o texto das mensagens e dá significado semântico a cada falha (login inválido, conta já existente, atributo não preenchido, auto-amizade, entre outras).

O encapsulamento das entidades é cuidadoso. A classe Usuario protege suas coleções internas, expondo apenas operações de alto nível e retornando cópias ou listas imutáveis (Collections.unmodifiableList) em vez das estruturas internas. O mesmo vale para Comunidade. Não há setters que exponham diretamente o estado interno.

A persistência foi isolada em um agregado serializável, a classe Dados, que reúne os repositórios de usuários e de comunidades em um único objeto. Dessa forma, a gravação e a leitura em disco preservam de forma atômica todo o estado necessário para sobreviver ao ciclo de encerrar e reabrir o sistema, exigido pelos testes de persistência (os pares _1/_2).

2.2 FRAQUEZAS DO PROJETO

Para facilitar o entendimento desta subseção, os comentários foram divididos pelos temas a que se referem.

HERANÇA E POLIMORFISMO

A principal limitação do design é o uso restrito de herança e polimorfismo de domínio. A única hierarquia rica é a de exceções. Os diferentes relacionamentos entre usuários — amizade, fã/ídolo, paquera e inimizade — são todos modelados como coleções de logins (Set ou List de String) dentro da classe Usuario. Para o domínio do Jackut essa escolha é idiomática e adequada, evitando abstrações artificiais; no entanto, em uma rubrica que valorize explicitamente polimorfismo, este é o flanco mais exposto do projeto.

PACOTE DE EXCEÇÕES

A separação das classes de exceção em um pacote próprio, distinto do pacote onde a exceção ocorre, cria um acoplamento entre pacotes que sempre é questionável quando a exceção não é genérica. Além disso, há um grau de granularidade elevado: várias exceções diferem apenas pela mensagem, como aquelas que sinalizam a tentativa de relação consigo mesmo (auto-amizade, auto-fã, auto-paquera, auto-inimigo). Há um argumento, válido em outros contextos, de que algumas dessas classes poderiam ser unificadas e parametrizadas pela mensagem. A decisão de manter classes separadas, neste projeto, atende ao requisito de mensagens exatas dos testes de aceitação e à legibilidade semântica, mas é uma troca consciente.

CONCENTRAÇÃO NA CLASSE USUARIO

A classe Usuario concentra grande parte do comportamento de domínio: perfil, amizades, recados, mensagens de comunidade, relações de fã/ídolo, paquera e inimizade. Embora cada responsabilidade esteja bem encapsulada em métodos coesos, a classe é extensa. Caso o sistema crescesse, parte desse comportamento poderia ser extraída para entidades ou objetos de valor próprios, reduzindo o tamanho da classe.

PERSISTÊNCIA POR SERIALIZAÇÃO

A persistência foi feita com serialização de objetos em arquivo binário, e não em um formato legível como XML ou JSON. Isso não é um problema funcional — atende plenamente

aos testes — mas apresenta uma ligeira desvantagem: os dados gravados não podem ser inspecionados diretamente abrindo o arquivo, dificultando a verificação manual do que foi persistido.

3. DIAGRAMA DE CLASSES E JUSTIFICATIVAS DE DESIGN

O diagrama a seguir apresenta uma visão simplificada das classes do Jackut, organizada pelas camadas da arquitetura. As cores distinguem as responsabilidades: em azul, a fachada e o orquestrador; em verde, os serviços de domínio; em laranja, os repositórios; em rosa, as entidades; e em amarelo, a persistência.

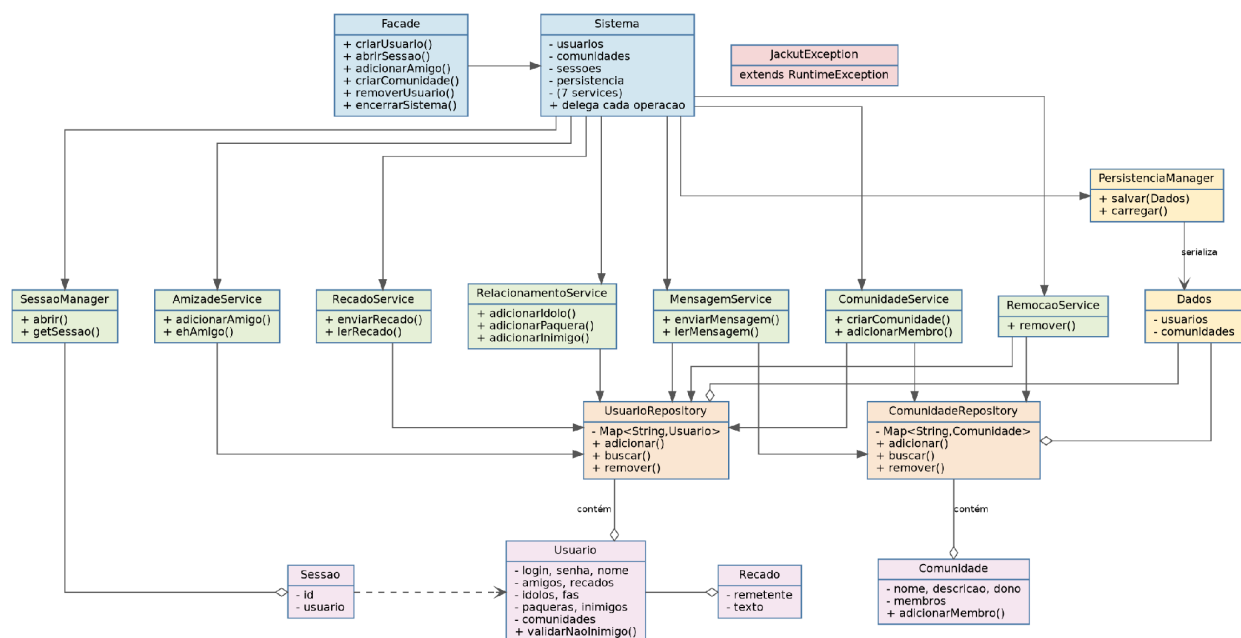


Ilustração 1 — Diagrama de classes simplificado do Jackut

As justificativas para as principais decisões de design são apresentadas a seguir.

Arquitetura em camadas. A escolha por separar fachada, orquestração, serviços, repositórios e modelos resolve a penalidade original de concentração de responsabilidades. Cada camada tem um motivo único para mudar: a Facade muda se a interface dos testes mudar; os serviços, se as regras de negócio mudarem; os repositórios, se a forma de armazenamento mudar. Isso reduz o acoplamento e torna a adição de funcionalidades — como ocorreu nas user stories 5 a 9 — uma tarefa localizada.

Separação entre Facade e Sistema. Poderia parecer redundante manter duas classes de coordenação, mas elas têm papéis distintos. A Facade adapta a interface esperada pelo EasyAccept (nomes de métodos e conversão de tipos); o Sistema resolve a sessão e o usuário de cada operação e encaminha ao serviço correto. Manter essa fronteira isola completamente a arquitetura interna do framework de testes.

Entidades ricas com encapsulamento forte. As regras que pertencem naturalmente a uma entidade ficam nela. O Usuario, por exemplo, sabe validar se há inimizade com outro usuário

(validarNaolnimigo) e gerencia suas próprias filas de recados e mensagens. As coleções internas nunca são expostas diretamente: retornam-se cópias ou listas imutáveis, protegendo a integridade do estado.

Agregado de persistência. Em vez de gravar usuários e comunidades em arquivos separados, ambos são reunidos no objeto serializável Dados. Isso garante que o estado seja gravado e recarregado de forma atômica, atendendo aos testes de persistência que exigem que usuários e comunidades sobrevivam juntos ao ciclo de encerrar e reabrir o sistema.

Hierarquia de exceções de domínio. Centralizar as falhas sob JackutException, com uma subclasse por regra de negócio, dá significado semântico a cada erro e centraliza as mensagens exigidas literalmente pelos testes, eliminando o uso de exceções genéricas.

4. DESIGN PATTERNS UTILIZADOS

Os padrões de projeto aplicados no Jackut, e os locais onde foram empregados, são descritos a seguir.

Facade (Fachada).

Aplicado na classe Facade. Ela oferece uma interface única e simplificada para o subsistema do Jackut, que é o ponto de entrada exigido pelo EasyAccept. A Facade não contém regra de negócio: cada método delega ao Sistema. O padrão isola o cliente (os scripts de teste) da complexidade interna da aplicação.

Repository (Repositório).

Aplicado nas classes UsuarioRepository e ComunidadeRepository. Cada uma encapsula a coleção de uma entidade e as regras de acesso e unicidade (login único, nome de comunidade único), oferecendo operações de alto nível (adicionar, buscar, remover, listar) e escondendo a estrutura de dados subjacente (um mapa indexado). Isso isola o restante do sistema dos detalhes de armazenamento.

Service Layer (Camada de Serviço).

Aplicado nas sete classes do pacote service. Cada serviço concentra as regras de uma área do domínio — AmizadeService para convites e amizades, RelacionamentoService para fã/ídolo, paquera e inimizade, RemocaoService para a remoção em cascata, e assim por diante. Esse padrão foi a peça central para resolver a penalidade de god class: a lógica que antes estava no Sistema foi distribuída entre serviços coesos.

Princípios GRASP — Expert e SRP.

Mais do que padrões, dois princípios guiaram a distribuição de responsabilidades. Pelo Information Expert, cada responsabilidade foi atribuída à classe que detém os dados necessários para cumpri-la: a validação de inimizade vive no Usuario, a gestão de membros na Comunidade, a serialização no PersistenciaManager. Pelo Single Responsibility Principle, cada classe tem um único motivo para mudar, o que sustenta a arquitetura em camadas descrita na seção anterior.

5. REFATORAMENTO E DESENVOLVIMENTO REALIZADO

O refatoramento entre as fases teve dois objetivos: corrigir as penalidades apontadas na avaliação do milestone 1 e preparar a arquitetura para as funcionalidades do milestone 2. As penalidades originais referiam-se ao tratamento inadequado de exceções, ao Sistema atuando como uma segunda fachada e como god class, e à fraca modularização. A seguir, detalha-se o que foi feito para cada user story e para cada penalidade.

Correção das penalidades (Milestone 1)

O tratamento de exceções foi inteiramente refeito. As RuntimeException genéricas, lançadas diretamente com mensagens soltas, foram substituídas por uma hierarquia de exceções de domínio sob a classe base JackutException. Cada regra de negócio passou a ter sua própria exceção, em arquivo separado, com a mensagem fixada no construtor.

Para resolver a crítica do Sistema como god class e como segunda fachada, a lógica de negócio foi extraída para uma camada de serviços. O Sistema deixou de varrer coleções e aplicar regras: passou a apenas resolver a sessão e delegar. Aplicou-se, assim, o princípio Expert, atribuindo cada responsabilidade à classe que detém os dados necessários — os repositórios para o armazenamento, os serviços para as regras, o SessaoManager para o ciclo de vida das sessões e o PersistenciaManager para a entrada e saída em disco.

A modularização foi reforçada com a separação em pacotes por responsabilidade e a inclusão de um arquivo package-info.java documentando cada pacote, atendendo também ao critério de documentação.

US5 e US6 — Comunidades

Foi introduzida a entidade Comunidade, identificada de forma única pelo nome, com descrição, dono e lista ordenada de membros, na qual o dono entra automaticamente como primeiro membro. A manipulação ficou a cargo do ComunidadeRepository e do ComunidadeService, mantendo sincronizadas a lista de membros da comunidade e a lista de comunidades de cada usuário.

US7 — Mensagens de comunidade

As mensagens enviadas a uma comunidade são entregues, em ordem, à caixa de mensagens de cada membro, por meio do MensagemService. Optou-se por manter essas mensagens independentes dos recados privados trocados entre usuários, com filas (FIFO) distintas dentro da classe Usuario.

US8 — Fã/ídolo, paquera e inimigo

Os novos relacionamentos foram concentrados no RelacionamentoService. A relação fã/ídolo é pública; a de paquera é privada e, quando correspondida, dispara um recado automático do sistema para ambos os usuários. Esse recado é gerado com remetente nulo, de modo que sobrevive à remoção de contas. A inimizade impede interações com o alvo e é verificada de forma bidirecional pelo próprio Usuario, no método validarNaoInimigo, acionado nas operações de adicionar amigo, adicionar ídolo, adicionar paquera e enviar recado.

US9 — Remoção de conta

A remoção de conta foi isolada no `RemocaoService`, responsável por apagar todos os rastros de um usuário em cascata: as comunidades das quais ele é dono são removidas, com a saída de todos os membros; sua participação em comunidades de terceiros é encerrada; os recados que enviou são purgados das caixas dos destinatários; e todas as referências a ele nas relações de outros usuários são limpas.

6. CONCLUSÃO

A experiência de refatorar o próprio código entre os milestones foi bastante válida, pois evidenciou, na prática, como a aplicação de padrões de projeto e de bons princípios de design facilita a manutenção e a extensão de um sistema. A separação em camadas e a distribuição de responsabilidades, motivadas pela correção das penalidades, aceleraram significativamente a implementação das funcionalidades do segundo milestone.

O resultado final é um sistema que compila sem erros no JDK 21 e passa integralmente na suíte de aceitação do EasyAccept — 18 arquivos de teste e 473 asserts aprovados, sem falhas, com a persistência validada pelos pares `_1/_2`. As quatro penalidades do milestone 1 foram neutralizadas, restando como ponto de evolução natural um maior uso de herança e polimorfismo de domínio, caso o sistema cresça.